

Claude Code

Masterclass

Construir, programar e automatizar com agentes de IA no terminal.

BASEADO EM

Curso de Nate Herk

AI AUTOMATION SOCIETY

ADAPTAÇÃO

Versão pt-PT · derivada com créditos

24 CAPÍTULOS

Da instalação à automação avançada

• ÍNDICE

O que *vais aprender.*

01 Porquê aprender isto •

02 Configuração inicial •

03 Operações •

04 Tokens / Contexto •

05 CLAUDE.md •

06 Primeiros workflows •

07 Publicar automações •

08 Arquitetura de projeto •

09 Comandos integrados •

10 RAG •

11 Construir / publicar sites •

12 APIs / MCPs •

13 Google CLI •

14 Assistente executivo próprio •

15 Skills •

16 Sub-agentes •

17 Equipas de agentes •

18 Automação de browser •

19 Permissões •

20 Gestão de contexto •

21 Github •

22 Worktrees •

23 Hacks divertidos •

24 Monetizar o teu conhecimento •

• INTRODUÇÃO

O que faz *o Claude Code*?

Um agente de IA que vive no terminal e escreve, depura e refatora código contigo — capaz de assumir um papel ativo no teu fluxo de trabalho diário.

A1

Apps, sites e automações

Construir produtos de ponta a ponta a partir de especificações.

A2

Workflows pessoais

Acelerar tarefas repetitivas e rotinas do dia a dia.

A3

Debug, refactor & testes

Analisar bases de código grandes e propor alterações cirúrgicas.

A4

Integrações com APIs

Ligar serviços externos com código pronto a produção.

A5

Geração de documentação

Documentar projetos, decisões e processos em segundos.

A6

Workflows de Git

Commits, branches, merges — sem sair da conversa.

PARTE I

I

Fundamentos operacionais.

Custo, prompting, modos de permissão, modelos, tokens, janela de contexto, ferramentas integradas e o ficheiro CLAUDE.md.

• O ARGUMENTO DO ROI

\$100/mês *versus* \$11.000/mês.

O salário médio de um engenheiro de software nos Estados Unidos ronda os **\$130.000 a \$139.000** por ano. O Claude Code, a **\$100 por mês**, consegue assumir uma fatia significativa do que esse profissional faz: escrever código, depurar, refatorar, gerar testes, criar documentação e gerir workflows de Git.

Quem usa Claude Code reporta ganhos de produtividade de **25 a 30%** no geral, com melhorias de **3 a 5×** em tarefas adequadas à assistência de IA.

10h/mês

TEMPO POUPADO POR PROGRAMADOR

\$1.000+

VALOR RECUPERADO

10×

RETORNO SOBRE O INVESTIMENTO

A MATEMÁTICA É SIMPLES

O ROI não é marginal — é transformacional.

• PORQUE O PROMPTING IMPORTA

A qualidade da *instrução* define a qualidade do *resultado*.

O Claude Code é um agente de IA. Um prompt vago produz resultados vagos. Um prompt estruturado e específico produz output limpo e fiável.

Pensa no Claude como um *contratante brilhante* que acabaste de contratar. Tem competência enorme, mas nunca viu o teu projeto. Quanto mais precisamente explicares o que queres — contexto, restrições e resultado esperado — melhor será o que recebes.

DOIS PRINCÍPIOS PRÁTICOS

“Devas estar 95% seguro antes de avançar.” · “Faz-me todas as perguntas necessárias para garantir que me compreendes.”

• MAU VS. BOM PROMPT

Específico bate *vago* — sempre.

● MAU PROMPT

“Constrói-me um website para o meu negócio de passear cães.”

● BOM PROMPT

“Cria uma landing page simples para um serviço de passeio de cães chamado ‘Happy Paws’. Deve ter um hero com um título, uma lista de três serviços com preços e um formulário de contacto no fundo. Estilo limpo e moderno, paleta azul e branco.”

→ Utiliza `plan mode` sempre que a tarefa não for trivial.

→ Deixa o Claude Code fazer-te as perguntas difíceis antes de começar a escrever código.

→ Itera no prompt antes de iterar no resultado.

• MODOS DE PERMISSÃO

Três níveis de *autonomia*.

MODO	O QUE FAZ	MELHOR PARA
Plan Mode	O Claude pode analisar e propor alterações, mas não executa nada. Modo só de leitura e pensamento.	Rever o que o Claude quer fazer antes de o fazer. Planeamento de arquitetura.
Accept Edits	O Claude lê, escreve e edita ficheiros sem pedir confirmação. Continua a pedir permissão para comandos de Bash/shell.	Fluxo de desenvolvimento normal — confias nas edições de ficheiros mas queres aprovar comandos.
Bypass Permissions	O Claude executa tudo sem perguntar: edições, comandos bash, operações de rede. Autonomia total.	Utilizadores experientes em ambientes controlados. Usar com cuidado.

RECOMENDAÇÃO

Em projetos críticos, fica em **Plan Mode** até confiar na proposta. Sobe para **Accept Edits** no dia-a-dia. Reserva **Bypass** apenas para tarefas autocontidas que possas reverter.

• MODELOS CLAUDE

Três tamanhos, *três trade-offs*.

MODELO	ALIAS	VERSÃO ATUAL	PONTOS FORTES	VELOCIDADE	CUSTO
Haiku	haiku	Haiku 4.5	Tarefas rápidas, simples, leves.	 0 MAIS RÁPIDO	 MAIS BARATO
Sonnet	sonnet	Sonnet 4.6	Programação do dia-a-dia, equilíbrio entre capacidade e custo.	  RÁPIDO	  INTERMÉDIO
Opus	opus	Opus 4.6	Raciocínio complexo, arquitetura, bugs difíceis.	 MAIS LENTO	   MAIS CARO

ESTRATÉGIA RECOMENDADA

Usa **Sonnet** em 80% do trabalho. Sobe para **Opus** em decisões de arquitetura ou bugs complicados. Volta a descer quando o problema estiver resolvido.

• COMO LER OS NÚMEROS DAS VERSÕES

Geração *major* · versão *minor*.

A Anthropic nomeia os modelos com um número de **geração** seguido de uma **versão** dentro dessa geração.

- 01

Claude 3 — família original de três escalões (Haiku, Sonnet, Opus), início de 2024.
- 02

Claude 3.5 — melhorias significativas em qualidade.
- 03

Claude 4 — programação de nível profissional, meados de 2025.
- 04

Claude 4.5 — mais eficiência e inteligência, Nov. de 2025.
- 05

Claude 4.6 — geração mais recente, início de 2026.

Como trocar de modelo

Usa `/model` durante a sessão, ou inicia com a flag:

```
# Iniciar uma sessão com Opus
claude --model opus
```

DICA DE FLUXO

Podes alternar modelos a meio de uma sessão sempre que quiseres. Não tens de fechar e reabrir nada.

• TOKENS

O que é *um token*?

Um **token** é a unidade fundamental que os modelos de IA leem e geram. Não corresponde exatamente a uma palavra — fica em torno de **3 a 4 caracteres**, ou aproximadamente **0,75 palavras** em inglês.

A palavra "authentication" parte-se em ~4-5 tokens:
[auth] [ent] [ic] [ation]

Código é mais "denso" em tokens que texto normal
por causa de caracteres especiais e sintaxe.

~4*chars*

≈ 1 TOKEN (MÉDIA)

~0,75*palavras*

≈ 1 TOKEN EM TEXTO INGLÊS

>1×

DENSIDADE DE TOKENS EM CÓDIGO

A memória *de trabalho* do Claude.

A janela de contexto é toda a informação que o Claude consegue manter e referenciar em simultâneo. Pensa nela como uma **secretária**: tudo o que o Claude está a usar tem de caber em cima dela.

→ O **system prompt** (instruções integradas)

→ Definições das ferramentas disponíveis

→ Os teus ficheiros `CLAUDE.md` (instruções de projeto)

→ Definições de servidores MCP (se existirem)

→ Todo o histórico da conversa

→ Ficheiros que o Claude leu durante a sessão

→ As respostas do próprio Claude

200.000 *tokens*

JANELA PADRÃO · ~500 PÁGINAS

1M *tokens*

MODELOS RECENTES (LONG CONTEXT)

IMPORTANTE

Quando se enche, perde-se contexto e qualidade. Gerir o que entra na “secretária” é uma competência por si só.

• PORQUE A GESTÃO DE CONTEXTO IMPORTA

Três *razões*.

- 01

Enche-se. À medida que a conversa cresce, consomes mais tokens. Sessões longas acabam por bater no limite.
- 02

O problema do “lost in the middle”. A informação no início e no fim da janela é priorizada; o que está a meio pode ser ignorado — tal como os humanos lembram-se melhor de princípios e fins.
- 03

Custo. Mais tokens significam mais custo computacional e consumo mais rápido da tua quota de subscrição.

• COMANDOS PARA GERIR CONTEXTO

Quatro comandos *essenciais*.

COMANDO	O QUE FAZ
<code>/context</code>	Mostra a utilização atual de tokens, partida por categoria — útil para identificar desperdício.
<code>/compact</code>	Comprime o histórico da conversa num resumo para libertar espaço. Podes especificar o que manter: <code>/compact retain the API design decisions</code> .
<code>/clear</code>	Apaga tudo e começa de novo. Usa quando mudas para uma tarefa completamente diferente.
<code>/rewind</code>	Volta a um ponto anterior da conversa e das alterações de código.

AUTO-COMPACTAÇÃO

O Claude Code comprime automaticamente a conversa quando te aproximas do limite — preserva decisões importantes ao mesmo tempo que liberta espaço.

• FERRAMENTAS INTEGRADAS

O *arsenal* do agente.

FERRAMENTA	O QUE FAZ	PEDE PERMISSÃO?
Read	Lê o conteúdo de um ficheiro. Pode ler intervalos específicos de linhas.	NÃO
Write	Cria um ficheiro novo ou sobrescreve completamente um existente.	SIM
Edit / MultiEdit	Edições cirúrgicas em partes específicas de um ficheiro, sem o reescrever inteiro.	SIM
Bash	Corre comandos de shell — <code>npm install</code> , <code>git commit</code> , <code>python script.py</code> .	SIM
Glob	Encontra ficheiros por padrão de nome/caminho (ex.: <code>**/*.ts</code>). É um “file finder”.	NÃO
Grep	Procura dentro dos ficheiros por padrões de texto/regex (ex.: <code>apiKey</code>).	NÃO
LS	Lista ficheiros e diretorias num dado caminho — como o <code>ls</code> do terminal.	NÃO
WebFetch	Obtém o conteúdo de um URL como referência.	SIM
WebSearch	Pesquisa na web.	NÃO

• DISTINÇÃO ESSENCIAL

Glob **vs.** Grep.

Confunde principiantes — vale a pena interiorizar a diferença.

● ● GLOB – ENCONTRA FICHEIROS

“Mostra-me todos os ficheiros `.tsx` dentro de `src/`.”

Procura por padrão de nome ou caminho.

● ● GREP – PROCURA CONTEÚDO

“Encontra todos os ficheiros que contêm a palavra `State`.”

Procura dentro do conteúdo dos ficheiros.

Memória de longo prazo *para o teu projeto.*

É um ficheiro Markdown que dá ao Claude instruções persistentes sobre o teu projeto. É carregado no início de *cada* sessão — sem ele, todas as sessões começam do zero.

Estrutura-o em três camadas:

01

O *Quê*

A tua stack técnica, estrutura do projeto, packages-chave.

02

O *Porquê*

O propósito de cada componente, decisões de arquitetura.

03

O *Como*

Como queres que o Claude trabalhe — padrões de código, convenções, workflows preferidos.

- INICIALIZAR UM PROJETO

O comando */init*.

Inicializa o teu projeto fazendo scan ao código existente e criando um `CLAUDE.md` automaticamente. Corre isto primeiro em qualquer projeto novo.

```
# Dentro da pasta do projeto:  
claude  
/init
```

BOA PRÁTICA

Depois do **/init**, revê e adapta o CLAUDE.md gerado. É um esqueleto — o valor está em o personalizares para o teu contexto real.

FIM DA PARTE I

*Parabéns.
Fundamentos fechados.*

Agora vamos construir os teus primeiros workflows agênticos com
Claude Code.

PARTE II

II

CLAUDE.md a sério, comandos & arquitetura.

Mais dicas para o teu CLAUDE.md, comandos slash essenciais e como organizar a pasta `.claude/`.

- O QUE UM BOM CLAUDE.MD INCLUI

Sete *secções*.

- 01

Visão geral do projeto — o que a aplicação faz.
- 02

Stack técnica — linguagens, frameworks, bases de dados.
- 03

Visão da arquitetura — como o código está organizado.
- 04

Convenções de código — naming, formatação, padrões.
- 05

Comandos comuns — como correr testes, fazer build, deploy.
- 06

Restrições — coisas que o Claude nunca deve fazer.
- 07

Onde encontrar mais — apontadores para contexto adicional.

• BOAS PRÁTICAS

Mantém-no *vivo*, curto e específico.

01

Abaixo de 200 linhas

É carregado em cada sessão e consome tokens. Mais curto = melhor cumprimento das regras.

02

Específico, não vago

“Usar indentação de 2 espaços” funciona melhor do que “formatar o código de forma agradável”.

03

Documento vivo

Atualiza-o à medida que o projeto evolui. Mete-o no controlo de versões para a equipa beneficiar.

04

Usa `/init`

Para gerar um esqueleto inicial baseado no teu código real.

05

Usa `.claude/rules/` para regras modulares

Em vez de meter tudo num ficheiro, divide por tópicos: `testing.md`, `api-design.md`, etc.

• ONDE COLOCAR O FICHEIRO

Três *localizações*, três âmbitos.

LOCALIZAÇÃO	ÂMBITO	PARTILHADO?
./CLAUDE.md ./.claude/CLAUDE.md	Este projeto (toda a equipa)	✓ SIM (VIA GIT)
~/.claude/CLAUDE.md	Todos os teus projetos (pessoal)	✗ NÃO
System-level paths	À escala da organização (gerido por IT)	✓ SIM

• AUTO MEMORY

Notas que *o Claude* escreve para si próprio.

Para além do `CLAUDE.md` (que tu escreves), o Claude Code tem **auto memory**: notas que o próprio Claude regista com base nas tuas correções e preferências.

Por exemplo, se disseses “*usa sempre pnpm, não npm*”, o Claude guarda essa instrução na sua memória.

COMANDO

Podes navegar e editar a auto-memory a qualquer momento com `/memory`.

```
# Localização padrão da auto-memory:
~/.claude/projects/
  <project>/memory/
    MEMORY.md

# As primeiras 200 linhas deste ficheiro
# são carregadas em cada sessão.
```


• GESTÃO DE SESSÃO

Comandos *slash* · sessão.

COMANDO	O QUE FAZ
<code>/init</code>	Inicializa o projeto fazendo scan ao código e criando um <code>CLAUDE.md</code> .
<code>/clear</code>	Apaga o histórico da conversa e começa do zero. Aliases: <code>/reset</code> , <code>/new</code> .
<code>/compact [foco]</code>	Comprime a conversa, preservando o que indicares. Ex.: <code>/compact retain the database schema and test results</code> .
<code>/rewind</code>	Restaura o código e a conversa a um ponto anterior. É um “undo” universal. Também acionado com Esc duas vezes.
<code>/resume</code>	Retoma uma sessão anterior por ID ou nome. As sessões ficam guardadas durante 30 dias.

• INFORMAÇÃO & DIAGNÓSTICO

Comandos *slash* · diagnóstico.

COMANDO	O QUE FAZ
<code>/context</code>	Mostra um detalhe visual da utilização da janela de contexto — quantos tokens estás a usar e onde vão.
<code>/cost</code>	Mostra a utilização de tokens e o custo da sessão atual.
<code>/model</code>	Troca de modelo a meio da sessão. Usa as setas para ajustar o nível de esforço.
<code>/help</code>	Lista todos os comandos disponíveis.
<code>/doctor</code>	Corre diagnósticos para verificar se a instalação do Claude Code está saudável.
<code>/status</code>	Mostra versão, modelo, conta e estado da ligação.

• CONFIGURAÇÃO

Comandos *slash* · configuração.

COMANDO	O QUE FAZ
<code>/memory</code>	Navega e edita os teus ficheiros de <code>CLAUDE.md</code> e de auto-memory.
<code>/config</code>	Abre o interface de definições.
<code>/permissions</code>	Vê ou atualiza quais as ferramentas autorizadas/negadas.
<code>/mcp</code>	Gere ligações a servidores MCP (Model Context Protocol).
<code>/agents</code>	Gere configurações de subagentes.

- ARQUITETURA DA PASTA DE PROJETO

Três *âmbitos* de configurações.

O Claude Code usa um sistema de configuração em camadas. Saber onde cada ficheiro vive — e quem afeta — é crítico.

ÂMBITO	LOCALIZAÇÃO	QUEM AFETA	PARTILHADO?
User (Global)	~/.claude/settings.json	Tu, em todos os projetos	NÃO
Project (Shared)	.claude/settings.json	Toda a equipa do projeto	✓ SIM (COMMITTED)
Local (Personal)	.claude/settings.local.json	Só tu, neste projeto	✗ NÃO (GITIGNORED)

- O QUE VAI ONDE

Regra *simples*: pessoal, equipa, local.

GLOBAL

`~/.claude/settings.json`

Defaults pessoais que queres em todo o lado — modelo preferido, tema, permissões pessoais.

PROJETO

`.claude/settings.json`

Standards de equipa — ferramentas permitidas, hooks partilhados, permissões. **É committed.**

LOCAL

`.claude/settings.local.json`

Overrides pessoais para este projeto — testes de config, definições específicas da máquina.
Gitignored.

• A PASTA .CLAUDE

Estrutura *canónica*.

```
.
├── CLAUDE.md
├── .claude/
│   ├── settings.json # equipa
│   ├── settings.local.json # pessoal
│   ├── rules/
│   │   ├── testing.md
│   │   └── api-design.md
│   ├── skills/
│   │   └── <skill>/SKILL.md
│   └── agents/
│       └── researcher.md
└── .mcp.json
```

O diretório global ~/.claude

A tua home também tem uma pasta `.claude/` para configuração pessoal cross-project. É lá que vivem os teus defaults, ficheiros de memória global e skills/agents pessoais.

PENSA NISTO COMO

O **.claude** do projeto é o do escritório. O **~/.claude** é o teu caderno pessoal — vai contigo onde quer que trabalhes.

• FICHEIROS E PROPÓSITOS

Quem faz *o quê*.

FICHEIRO	PROPÓSITO	PARTILHADO?
CLAUDE.md	Instruções de projeto carregadas em cada sessão.	✓ SIM
.claude/settings.json	Configuração partilhada da equipa (permissões, hooks).	✓ SIM
.claude/settings.local.json	Configuração pessoal para este projeto.	✗ NÃO
.claude/rules/*.md	Regras modulares, opcionalmente com escopo por tipo de ficheiro.	✓ SIM
.claude/skills/*/SKILL.md	Skills personalizadas que o Claude auto-descobre.	✓ SIM
.claude/agents/*.md	Personas personalizadas de subagentes.	✓ SIM
.mcp.json	Ligações a servidores MCP com escopo de projeto.	✓ SIM

• O QUE É O .GITIGNORE

A “lista *não rastrear isto*” do projeto.

O **Git** é o sistema que regista as alterações ao teu código ao longo do tempo. O **.gitignore** é apenas um ficheiro de texto onde listas ficheiros e pastas que o Git deve fingir que não existem.

Tipicamente ignoras:

- `node_modules/` e outras pastas de dependências
- Pastas de build como `dist/` ou `build/`
- Ficheiros de segredos como `.env` com chaves de API
- Lixo de editor/OS como `.vscode/`, `.DS_Store`, `Thumbs.db`

Se está no .gitignore, o Git:

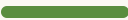
- 01 Não mostra esses ficheiros como “alterados”.
- 02 Não os inclui nos commits.
- 03 Trata-os como ruído que não queres partilhar.

EM RESUMO

É a lista “*não rastrear isto*” do teu projeto.

• CORES DOS FICHEIROS NO EDITOR

A semântica das *cores*.



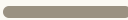
VERDE

Ficheiro novo que o Git nunca viu.
Untracked — ainda não foi committed.
Após o commit, volta à cor normal.



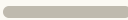
AMARELO / LARANJA

Ficheiro já conhecido pelo Git, mas com **alterações** desde o último commit. Volta à cor normal depois de o commits.



NORMAL

Ficheiro rastreado e **sem alterações** face ao último commit.



CINZENTO

Existe no projeto mas está listado em `.gitignore`.

FIM DA PARTE II

Parabéns.
Voltamos a construir.

Bases lançadas — agora vamos ao tema que separa quem usa o
Claude Code de quem o domina: gestão de contexto.

PARTE III

III

Gerir o *contexto*.

Oito práticas para tirar mais da janela de contexto sem desperdiçar tokens nem degradar a qualidade do output.

• PORQUÊ ESTE CAPÍTULO

O *gargalo* oculto.

O Claude Code brilha em pair programming e em workflows agênticos — mas a sua eficácia depende de uma gestão inteligente do contexto. A janela de contexto enche-se rapidamente com leituras de ficheiros, chamadas a ferramentas e histórico da conversa.

Isso leva a compactações, desperdício de tokens e qualidade de output degradada. As próximas oito secções são práticas concretas para evitar esse colapso.

Mantém o *CLAUDE.md* enxuto e focado.

Coloca este Markdown na raiz do projeto (ou em subpastas) para instruções persistentes — stack, features-chave, regras inegociáveis. Limita-o a **~200 linhas** para não inchar todas as sessões.

TRUQUE

Move documentação de referência detalhada para **skills** on-demand. O CLAUDE.md fica para o essencial; o resto carrega-se quando for preciso.

Usa */clear* e */compact* estrategicamente.

Corre `/clear` para limpar o histórico depois de tarefas grandes ou quando mudas de tópico — **mas cria um documento de handoff primeiro** para preservar a informação chave.

Usa `/compact` para resumir e reter o essencial a meio da sessão. Especifica o que manter: `/compact keep logic`. Isto evita que a auto-compactação interrompa o teu trabalho.

```
# Padrão recomendado:  
1. Termina a tarefa  
2. /compact retain <decisões essenciais>  
3. Escreve handoff.md  
4. /clear  
5. Próxima tarefa começa limpa
```

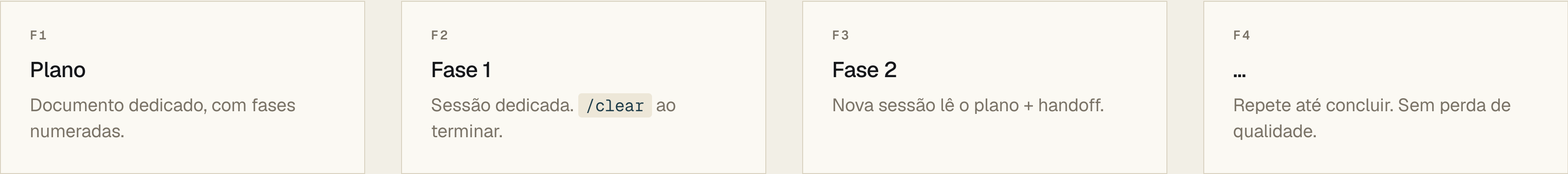
Visualiza o teu *consumo* de contexto.

Escreve `/context` para uma quebra do consumo de tokens por skills, MCPs, ficheiros e histórico. Ajuda a identificar desperdício — como chamadas desnecessárias a ferramentas — antes de bateres no limite.

```
# Exemplo de output (resumo):  
System prompt           3.412 tokens  
Tool definitions         8.901 tokens  
CLAUDE.md               1.200 tokens  
Files read               42.110 tokens  
Conversation history     38.755 tokens  
-----  
Total used               94.378 / 200.000
```

Planeia *upfront*, trabalha em fases.

Cria um plano detalhado guardado num ficheiro, depois constrói por etapas. Limpa sessões entre fases — ou deixa o Claude rastrear o progresso. Isto evita overflow de contexto em projetos longos.



Limita as *sessões abertas*.

Não malabarismos com mais de **3 a 4 sessões paralelas** para evitar contexto fragmentado. Para multi-tasking, usa ficheiros de handoff ou ficheiros de memória partilhada com timestamps para os agentes consultarem.

3–4 SESSÕES PARALELAS MÁXIMAS	1 FICHEIRO DE HANDOFF POR TROCA DE FASE	∞ MEMÓRIA PARTILHADA TIMESTAMP-KEYED
---	---	--

Sub-agentes para tarefas isoladas.

Delega investigações pesadas em ficheiros (ex.: ler 3+ ficheiros) a **sub-agentes** que correm em contextos separados.

Eles devolvem apenas o resumo, mantendo a tua sessão principal limpa. Acrescenta regras ao teu CLAUDE.md como “*Spawn subagent for multi-file analysis.*”

Isto transforma o Claude numa **equipa de investigação** e reduz drasticamente o consumo de tokens.

```
# Em CLAUDE.md:  
## Subagent rules  
- Spawn a subagent for any task  
  requiring 3+ file reads.  
- Subagent returns a summary,  
  not raw file contents.  
- Main session stays under  
  50k tokens.
```

Memória partilhada para multi-agentes.

Instruí os agentes a escreverem memórias num único ficheiro com timestamps. Isto força a partilha de contexto entre instâncias sem duplicação.

```
# shared-memory.md
```

```
/// 2026-03-11 14:02 – agent: researcher
```

```
Identifiquei 3 ficheiros relevantes para a feature X.
```

```
Resumo dos pontos-chave: ...
```

```
/// 2026-03-11 14:15 – agent: builder
```

```
Implementei a função baseada no resumo do researcher.
```

```
Próximo passo: testes.
```

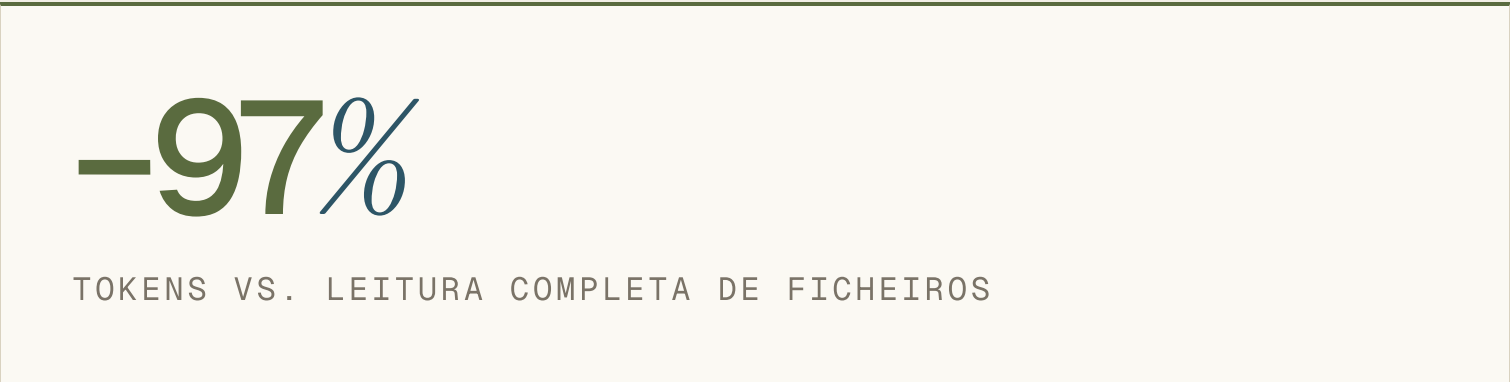
```
/// 2026-03-11 14:38 – agent: tester
```

```
3 testes a passar, 1 a falhar (edge case com null).
```

Pesquisa semântica local *em vez de grep.*

Em bases de código grandes, usa ferramentas de **pesquisa semântica local** para consultar ficheiros *sem* carregar tudo para contexto.

Benchmarks mostram **redução de 97%** no consumo de tokens face a leituras completas.



QUANDO APLICAR

Qualquer projeto onde leitura indiscriminada por `Read` ou `Grep` ultrapassa **50 ficheiros**.

PARTE IV

IV

GitHub & Worktrees.

Onde vivem os teus ficheiros — e como correr várias sessões do Claude Code em paralelo sem que se atropelem.

- PORQUE IMPORTA

Duas peças *fundacionais*.

Antes de mergulhar nas funcionalidades avançadas do Claude Code, é preciso perceber duas coisas: o **GitHub** (onde os ficheiros do teu projeto vivem e são rastreados) e os **Git Worktrees** (uma forma de trabalhar em várias coisas em simultâneo sem partir nada).

Estes dois conceitos desbloqueiam uma das capacidades mais poderosas do Claude Code: correr *múltiplas sessões de programação com IA em paralelo*, cada uma a trabalhar em tarefas separadas, sem pisarem os pés umas às outras.

• O QUE É O GIT

Uma *máquina do tempo* para o teu projeto.

O Git é um **sistema de controlo de versões** — uma ferramenta que rastreia cada alteração feita aos teus ficheiros ao longo do tempo.

Sempre que guardas uma alteração significativa (chamada **commit**), o Git tira uma fotografia do projeto inteiro nesse momento. Se algo partir mais tarde, podes rebobinar para qualquer fotografia anterior e recomeçar dali.

ANALOGIA

Como um **save** de videojogo: cada commit é um ponto de gravação a que podes voltar sempre.

• CONCEITOS-CHAVE EM LINGUAGEM SIMPLES

Repo, *Commit*, Branch.

01

Repository (*Repo*)

Pasta de projeto que o Git está a observar. Contém todos os ficheiros e o histórico completo. É como uma conta bancária para o teu código — guarda tudo e rastreia cada transação.

02

Commit

Uma fotografia do projeto num momento específico. Cada commit inclui uma mensagem a descrever o que mudou e porquê. Como carregar em “save” num videojogo.

03

Branch

Uma cópia separada do projeto onde podes fazer alterações sem afetar a versão principal. O branch padrão chama-se normalmente `main`. Ao criar `feature-login`, estás a dizer: “deixa-me experimentar sem arriscar a versão estável.”

• CONCEITOS-CHAVE EM LINGUAGEM SIMPLES

Push, *Pull*, Merge.

04

Push

Carregar os teus commits locais para a versão online do repositório. Como sincronizar para a cloud.

05

Pull

Descarregar as alterações mais recentes do repositório online para a tua máquina local.

06

Merge

Combinar as alterações de um branch noutro. Quando a tua feature está pronta e testada, fazes merge de volta para `main`.

- O QUE É O GITHUB

O *parque de estacionamento* dos teus projetos.

O GitHub é um site e serviço cloud construído por cima do Git. Enquanto o **Git** corre no teu computador local e rastreia os teus ficheiros, o **GitHub** aloja os teus repositórios online — para os acederes em qualquer lado, fazeres backup e colaborares.

DIFERENÇA EM UMA FRASE

O Git é o **motor** que rastreia as tuas alterações. O GitHub é o **parque de estacionamento na cloud** onde o teu projeto vive em segurança e ao alcance de qualquer pessoa autorizada.

- O QUE O GITHUB TE DÁ

Quatro *benefícios*.

01

Backup na cloud

Os ficheiros estão online — se o computador morrer, o código está salvo.

02

Colaboração

Várias pessoas podem trabalhar no mesmo projeto sem sobrescrever o trabalho umas das outras.

03

Pull Requests

Forma de propor alterações e tê-las revistas antes de fazerem merge para o projeto principal — o coração da colaboração.

04

Histórico de versões

Vês cada alteração feita ao projeto, quem a fez e quando.

• O WORKFLOW BÁSICO DO GITHUB

Sete *passos*.

- 01

Criar um **repositório** no GitHub para o projeto.
- 02

Clonar (copiar) para o computador local.
- 03

Criar um **branch** para a nova feature ou correção.
- 04

Fazer alterações e **commit** (guardar snapshots).
- 05

Push do branch para o GitHub.
- 06

Abrir um **Pull Request** a propor o merge para `main` .
- 07

Rever, aprovar e fazer **merge**.

É COMO OS PROFISSIONAIS TRABALHAM

Este é exatamente o workflow que developers e equipas usam, e é o mesmo que o Claude Code utiliza por trás quando gere o teu projeto.

Vários *branches* abertos ao mesmo tempo.

Um **Git worktree** permite-te ter vários branches do mesmo repositório com *checkout simultâneo*, cada um na sua pasta separada do computador.

```
# Criar uma worktree para o branch hotfix-bug
git worktree add ../meu-projeto-hotfix hotfix-bug

# Agora tens duas pastas:
# ./meu-projeto (branch: feature-x)
# ../meu-projeto-hotfix (branch: hotfix-bug)
```


• O PROBLEMA QUE WORKTREES RESOLVEM

Sem worktrees — o *context-switching* doloroso.

Normalmente, no Git, só consegues trabalhar num branch de cada vez por pasta. Se estiveres a construir uma feature nova e de repente surgir um bug urgente, sem worktrees terias de:

- 01
- Parar o que estás a fazer.
- 02
- Stash** (guardar temporariamente) todo o trabalho atual.
- 03
- Mudar para outro branch.
- 04
- Corrigir o bug e fazer commit.
- 05
- Voltar ao branch original.
- 06
- Restaurar o stash e continuar.

O CUSTO

Muito context-switching, muita margem para erros — e o ritmo de trabalho parte-se ao meio.

• COM WORKTREES

Duas *secretárias* no mesmo escritório.

Com worktrees, crias simplesmente um **segundo diretório de trabalho** para o branch do bug. O teu trabalho original fica exatamente onde o deixaste, intocado, na sua própria pasta. Ambas partilham o mesmo histórico Git.

SECRETÁRIA 1

Feature em desenvolvimento

Os teus rascunhos, ficheiros abertos e tudo o que estavas a fazer — exatamente como o deixaste.

feature-x.tsx

tests/

notes.md

SECRETÁRIA 2

Hotfix urgente

O bug a corrigir, no seu próprio espaço. Nenhum dos lados perturba o outro.

hotfix-bug.ts

changelog.md

Ambas as secretárias fazem parte do mesmo escritório (o mesmo repositório), mas cada uma tem o seu espaço de trabalho. Passas entre elas sem ter de arrumar nada.

- WORKTREES + CLAUDE CODE

Onde tudo *se junta*.

O Claude Code tem suporte nativo para Git worktrees através da flag `--worktree` (ou `-w`). Significa que podes correr **múltiplas sessões do Claude Code ao mesmo tempo**, cada uma a trabalhar numa tarefa completamente diferente, sem se atropelarem.

```
# Sessão 1 – feature
claude --worktree feature-login

# Em paralelo, noutro terminal:
# Sessão 2 – hotfix
claude -w hotfix-bug

# Sessão 3 – exploração
claude -w spike-redesign
```

A CAPACIDADE

Três tarefas a avançar em simultâneo. Três contextos isolados. Zero colisões.

PARTE V

V

Cinco superfícies, o *mesmo motor*.

Terminal, app de desktop, web, IDE ou servidor remoto — escolher bem onde corres o Claude Code muda completamente o teu fluxo de trabalho.

• VISÃO GERAL

As *cinco* formas de correr.

Todas usam o mesmo motor Claude Code por baixo. O que muda é a interface, o nível de controlo e onde a sessão vive.

01

Terminal

CLI puro. Máximo poder, máxima hackabilidade.

02

Desktop App

GUI visual, com pré-visualização e tarefas agendadas.

03

Web

`claude.ai/code` · corre na cloud, mesmo offline.

04

IDE (VS Code)

Extensão dentro do editor — zero context-switching.

05

VPS

Servidor próprio, sempre ligado. Cloud Code no bolso.

NÃO TENS DE ESCOLHER UMA SÓ

O criador do Claude Code, **Boris Cherny**, corre 5 sessões no terminal e 5–10 na web em paralelo. Combina as superfícies em função da tarefa.

• 01 – TERMINAL

O *fundamento*.

Abres o terminal, escreves `claude`, e estás em conversa. Lê ficheiros, edita pastas, executa código. Tudo texto — sem botões, sem menus.

Todas as outras superfícies são *wrappers* deste motor. Aqui chegam primeiro as **features novas** e os **comandos avançados**.

```
# Instalar (ver docs/quickstart)
npm install -g @anthropic-ai/claude-code

# Iniciar dentro de qualquer pasta
cd meu-projeto
claude
```

PRÓS

Máximo controlo

Todos os comandos, todas as flags. Funciona com qualquer editor.

CONTRAS

Curva de aprendizagem

Só texto — pode ser intimidante se nunca usaste terminal.

PARA QUEM

Já vives na CLI

Se "terminal" não te assusta, este é o teu lar.

O *GUI* oficial.

Inclui **Chat**, **Co-work** e **Claude Code** num só sítio. Vês as alterações linha a linha e podes aceitar/rejeitar com botões.

Tem **app preview** integrado: se estás a construir um site ou app, arranca o dev server e mostra-o dentro da própria app.

Inclui também **scheduled tasks** — tarefas que correm enquanto a app está aberta (diárias, semanais).

PRÓS

Visual e amigável

Sessões em paralelo, isoladas em branches. Menos intimidante.

CONTRAS

Menos hackável

Releases por vezes atrás da CLI. Só Mac e Windows.

PARA QUEM

Gostas de clicar

Já usas o app do Claude e queres adicionar Code sem ir ao terminal.

Claude Code na *cloud*.

Sem instalação local. Vai a `claude.ai/code`, liga o teu repositório GitHub, e o Claude clona-o num ambiente cloud e trabalha lá.

Crítico: **continua a correr mesmo com o computador desligado**. Ideal para arrancar uma tarefa demorada e voltar mais tarde — também acessível do telemóvel ou iPad.

WORKFLOW TÍPICO

Conectas repo → atribuíς tarefa → fechas o portátil → voltas e revês o pull request.

PRÓS

Sempre ligado

Sessões persistem. Trabalho contínuo de qualquer dispositivo.

CONTRAS

Research preview

Experimental — exige repositório GitHub.

PARA QUEM

Tarefas longas

Dar tarefa e seguir vida. Programar do iPad.

• 04 – IDE · VS CODE, CURSOR, ANTI-GRAVITY

O *melhor dos dois mundos*.

A extensão do Claude Code corre dentro do **VS Code** (gratuito), **Cursor** ou **Anti-gravity**. O Claude vê o que tu vês no editor — zero context-switching.

Status indicators no topo: **ponto azul** = Claude está à tua espera; **ponto laranja** = Claude terminou.

Podes ter **split-pane** com várias sessões a correr em paralelo, abrir o terminal no mesmo painel para comandos como `/agents`, e deixar comentários inline em planos.

```
VS Code
├─ Files panel
│   └─ CLAUDE.md
│       └─ workflows/
│           └─ tools/
├─ Editor (centro)
└─ Claude Code panel (direita)
    └─ Plan visível
        └─ Status: ● a esperar
            └─ Terminal embutido
```

RECOMENDADO PARA 90% DOS CASOS

Se hesitas, começa aqui. Está onde já passas o dia.

O Claude Code *no bolso.*

Corre o Claude Code num **servidor privado virtual** (ex.: Hostinger, ~6 €/mês). Como vive num servidor remoto, está **sempre ligado**.

Podes fazer **SSH** a partir do teu terminal, VS Code ou app. Combinado com uma ponte (ex.: Telegram), tens Claude Code acessível do telemóvel — onde quer que estejas.

ATENÇÃO A PERMISSÕES

Num servidor, o Claude tem acesso real à infraestrutura. Limita as permissões conscientemente.

PRÓS

Acesso à infraestrutura

Docker, bases de dados, automações — tudo lado a lado. Cross-device.

CONTRAS

Mais setup

Requer conhecimentos básicos de servidor. Mais fricção inicial.

PARA QUEM

Operações 24/7

Corres servidores, deploys, ou queres um assistente sempre ligado.

PARTE VI

VI

Workflows
agênticos.

Como organizar o trabalho do Claude Code com o framework WAT — Workflows, Agent, Tools. A base para construir o teu primeiro workflow.

• DUAS PALAVRAS-CHAVE

Determinístico **vs.** não-determinístico.

● ● DETERMINÍSTICO

*Dado um input, sabes **exatamente** qual será o output. Previsível. Aborrecido — no bom sentido.*

Exemplo: enviar email, gerar PDF, query a uma base de dados.

● ● NÃO-DETERMINÍSTICO

*O output varia — há **juízo, IA, criatividade**. Útil onde a tarefa é demasiado variável para regras fixas.*

Exemplo: redigir um resumo, classificar um lead, decidir um próximo passo.

O NOSSO TRABALHO COMO CONSTRUTORES

Tornar um processo *não-determinístico* tão *determinístico* quanto possível — sem o reduzir a um script frágil.

• WAT

Três camadas, *uma estrutura*.

Sem estrutura, o trabalho do Claude vira o cacifo da escola — tudo enfiado lá dentro, impossível de encontrar nada. O WAT dá-lhe pastas, separadores e etiquetas.

W · WORKFLOWS

As instruções

Ficheiros Markdown a definir **processos**: objetivo, inputs, ferramentas a usar, output esperado, edge cases. Linguagem natural, como um SOP. Vivem em `workflows/`.

A · AGENT

O coordenador

O próprio Claude Code. Lê o workflow, escolhe que ferramentas chamar, gere a sequência, trata de erros, pede clarificações. **Project manager** da operação.

T · TOOLS

A execução

Scripts Python (ou outros) que **fazem** uma coisa concreta: scraping, gerar PDF, query a uma API. Modulares — reutilizáveis em vários workflows. Vivem em `tools/`.

PORQUÊ SEPARAR

A IA não-determinística trata do *raciocínio*; o código determinístico trata da *execução*. Esta separação é o que torna o sistema fiável.

• COMO FICA NA PASTA

Anatomia de um *projeto WAT*.

```
meu-workflow/
├── CLAUDE.md # onboarding do agente
├── .env # segredos · API keys
├── workflows/
│   ├── competitor_analysis.md
│   └── daily_digest.md
├── tools/
│   ├── scrape_site.py
│   ├── analyze_findings.py
│   └── generate_pdf.py
└── temp/ # outputs intermédios
```

A regra de ouro dos segredos

As API keys **nunca** vão para o código das ferramentas. Vão para `.env`, que entra no `.gitignore`.

```
# .env
FIRECRAWL_API_KEY=fc_xxx...
OPENAI_API_KEY=sk_xxx...
TAVILY_API_KEY=tvly_xxx...
```

AUTO-MELHORIA

Quando uma ferramenta falha (rate limit, endpoint mudou), o agente lê o erro, corrige a tool, e **atualiza o workflow** para que o erro não volte.

• EXEMPLO · ANÁLISE DE CONCORRÊNCIA

Como as *três camadas* trabalham juntas.

Pedido: *"Pesquisa o preço da empresa X e cria-me um relatório PDF."* O agente toma todas as decisões.

01 Lê o **workflow** de análise de concorrência.

02 Chama `web_search` para encontrar URLs.

03 Chama `scrape_site` para extrair conteúdo.

04 Chama `analyze_findings` para sintetizar.

05 Chama `generate_pdf` para o relatório.

A MARAVILHA

Não mapeaste passo a passo. Deste objetivo, ferramentas e output — o agente fez o resto, e *aprende* com feedback.

```
# workflows/competitor_analysis.md
```

Objetivo

Analisar preço, posicionamento
e claims de um concorrente.

Inputs

- URL ou nome do concorrente

Ferramentas

- web_search
- scrape_site
- analyze_findings
- generate_pdf

Output

PDF em `reports/<data>.pdf`

PARTE VII

VII

Skills, sub-agents & agent teams.

Onde o Claude Code passa de "ferramenta produtiva" a "equipa virtual". Três conceitos que multiplicam a alavancagem.

• SKILLS

Instruções *reutilizáveis*.

Uma **skill** é um conjunto de instruções que escreves uma vez, guardas, e podes invocar a qualquer momento. Resultado: muito mais *consistência* porque o processo é sempre o mesmo.

PRODUTIVIDADE PESSOAL

"Morning coffee"

Olha agenda, ClickUp, tarefas da semana — propõe-te o plano do dia. *Zero decision fatigue*.

CONTEÚDO

"Carousel skill"

Pega na transcrição de um vídeo e gera 8 slides em formato tweet/LinkedIn.

VISUAL

"Excalidraw visuals"

Cria diagramas Excalidraw com texto perfeito — nada de letras de IA mal formadas.

TRÊS RAZÕES PARA TE IMPORTAREM

1. Produtividade pessoal · automatizas o que repetes. **2. Alavancagem de equipa** · transformas SOPs em automações partilhadas. **3. Monetização** · skills úteis podem ser vendidas, num momento em que ainda há janela.

• SUB-AGENTS

Trabalhadores *focados*, em paralelo.

Um **sub-agent** dorme até a sessão principal o "acordar" com um prompt. Acorda com *contexto fresco*, modelo próprio, ferramentas próprias e um objetivo concreto. Devolve o resumo e desaparece.

A sessão principal vira **project lead**: planeia, delega, recebe resultados — sem poluir o seu contexto com tarefas especializadas.

A ANALOGIA

Para uns cupcakes bons não vais ao supermercado — vais a uma pastelaria de nicho. Sub-agents são pastelarias especializadas.

Cinco razões para usar

- 01

Preservar contexto — main thread limpa.
- 02

Limitar permissões — sub-agent com tools restritas para tarefas arriscadas.
- 03

Reutilizar configs — partilhar entre projetos e equipa.
- 04

Especialização — agentes treinados para nichos dão melhor output.
- 05

Paralelismo — várias tarefas a correr em simultâneo.

• DISTINÇÃO CRÍTICA

Sub-agents **vs.** Agent Teams.

● ● SUB-AGENTS

Trabalhadores focados que correm *em paralelo*, mas não comunicam entre si. Recebem prompt do main agent → trabalham → devolvem.

Relação unidirecional · main ↔ worker.

● ● AGENT TEAMS

Partilham uma *task list*. Comunicam entre si. Podem atribuir tarefas uns aos outros. Verdadeiro ecossistema cooperativo.

Relação multidirecional · agent ↔ agent.

REGRA PRÁTICA

Tarefas **isoladas** em paralelo → sub-agents. Tarefas que precisam de **coordenação** entre especialistas → agent teams.

• AGENT TEAMS

Quando os *especialistas* falam entre si.

Dás uma tarefa multi-fase: *"Refactor X, Y, Z e acrescenta testes."*

A main session ouve, procura agentes especializados, atribui:

- **Refactor agent** → reescreve o código.
- **Test writer agent** → cria os testes.
- Os dois **comunicam** à medida que avançam.
- Devolvem resultados consolidados à main.

```
# shared-tasks.md (memória partilhada)

## Task 1 – refactor auth.ts
- owner: refactor-agent
- status: ✅ done
- handoff: tests needed for new
  validateToken() signature

## Task 2 – write tests
- owner: test-writer
- status: 🔄 in progress
- depends_on: Task 1

# Cleanup graceful no fim
# em vez de force-kill
```

PARTE VIII

VIII

APIs, MCPs e browser automation.

Como dar superpoderes ao Claude Code — trazer dados em tempo real, ligar serviços externos e controlar um browser real.

• MODEL CONTEXT PROTOCOL

O que é um *MCP server*?

O Claude Code é inteligente, mas o seu mundo é o que tu lhe mostras. Um **MCP server** é uma forma padronizada de ligar o agente a serviços externos — APIs, bases de dados, ferramentas — sem teres de fazer chamadas HTTP à mão.

O que ganhas

- O Claude descobre automaticamente as tools que o MCP expõe.
- Decide qual usar e em que ordem, em *linguagem natural* tua.
- Trata erros e *fallbacks* sozinho.

```
# Ligar um MCP server (Firecrawl)
# a partir do terminal:
claude mcp add firecrawl \
  --env FIRECRAWL_API_KEY=$KEY

# Listar MCP servers ativos:
/mcp
```

ATENÇÃO AO TOKEN-BLOAT

Cada MCP ativo carrega definições de tools que **consomem tokens**. Vê o impacto com `/context` e desliga os que não estás a usar.

Qualquer site → dados *LM-ready*.

O **Firecrawl** expõe quatro operações via MCP: *scrape* (uma página), *map* (todos os URLs), *crawl* (explorar tudo) e *search* (procurar + extrair). O Claude decide qual chamar com base no teu pedido em linguagem natural.

01 Scrape Markdown, screenshot, branding, links, JSON estruturado de uma página.	02 Map Listar todos os URLs de um site — perceber a arquitetura.	03 Crawl Percorrer várias páginas em sequência, recursivamente.	04 Search Pesquisa web + scraping das páginas relevantes.
---	---	--	--

BOA PRÁTICA

Em vez de meter a documentação completa do MCP no `CLAUDE.md`, cria uma **cheat sheet** à parte (ex.: `firecrawl-cheatsheet.md`) e referencia-a. Mantém o `CLAUDE.md` leve.

Dar ao Claude *um browser real.*

Com **Playwright CLI**, o Claude pode abrir páginas, tirar screenshots, preencher formulários, clicar botões — automatizar fluxos sem API.

Porquê Playwright CLI em vez do MCP de Chrome DevTools? Os MCPs grandes "comem" tokens com as suas definições. A CLI executa scripts sob comando, sem inchar contexto.

```
# Em plan mode, primeiro:  
"Quero usar Playwright CLI para  
automação de browser. Investiga,  
instala, cria um script demo."  
  
# Depois /init para criar CLAUDE.md
```

Três casos de uso

01

QA automático
Constrói app → testa app → encontra bugs → corrige-se.

02

Lead scraping
Procurar empresas (ex.: dentistas) e extrair contactos.

03

Sessões autenticadas
Operar dentro de plataformas onde precisas de estar logado.

PARTE IX

IX

Hacks que tiram o *máximo*.

Pequenos hábitos e comandos que separam quem usa o Claude Code de quem realmente o domina. Princípio universal: plan first, ask questions, keep context small.

• INICIANTE

Seis *hacks* para começar bem.

01 · ONBOARDING

Corre sempre `/init`

Em qualquer projeto novo. O Claude faz scan e cria um CLAUDE.md adaptado.

02 · VISIBILIDADE

Status line

`/statusline` · adiciona modelo, % de contexto e custo no fundo do terminal. Mini-dashboard sempre à vista.

03 · VELOCIDADE

Voice input

Comando `/v` nativo, ou ferramenta de voice-to-text. Falas mais depressa do que escreves.

04 · HIGIENE

Mantém o contexto pequeno

Não despejes a codebase. Só o que é preciso para a tarefa atual.

05 · DIAGNÓSTICO

Usa `/context`

Vê exatamente onde estás a queimar tokens. Reestrutura.

06 · PLAN MODE

Começa sempre em plan mode

Shift+Tab alterna modos. Plano primeiro, código depois — reduz drasticamente as voltas.

• POWER USER

Quatro *hacks* que mudam o jogo.

- 01

Compact a 60%, clear entre tarefas — `/compact keep API integration decisions` mantém o essencial enquanto liberta espaço.
- 02

Trata o Claude como junior developer — não dês comandos diretos, dá-lhe *problemas*. "Como devíamos lidar com X?" produz melhor output do que "Escreve uma função que faça X".
- 03

Força perguntas — diz-lhe explicitamente *"Continua a perguntar-me até estares 95% seguro do que preciso."* Invoca a ferramenta `ask_user_question`.
- 04

Curiosidade genuína — lê o que o agente está a pensar e os comandos que executa. Faz perguntas. Pede exercícios. O Claude é o teu melhor professor.

MENTALIDADE

O Claude Code foi desenhado para ser **insanamente customizável e hackável**. Se te perguntas se algo é possível, pergunta-lhe a ele.

PARTE X

X

Monetizar o teu *conhecimento*.

Como transformar competência em Claude Code num negócio — vender resultados em vez de tecnologia, preço pelo valor e construir relações de longo prazo.

• MENTALIDADE

Para de vender *IA*. Vende *resultados*.

As empresas só se preocupam com três coisas: **tempo, dinheiro, foco**. Não querem saber se entregas com IA, com um VA, ou com fita-cola e pastilha elástica — querem chegar ao destino, rápido, barato e sem stress.

● DISCURSO "CENTRADO NA IA"

"Construí um workflow com 15 nodes, chamadas a 4 APIs e arquitetura multi-agente."

● DISCURSO "CENTRADO NO RESULTADO"

*"Vai poupar-te 10 horas por semana em criação de conteúdo." — **foi assim que vendi o meu primeiro workflow a \$1.200.***

A REFORMULAÇÃO

Em vez de *"bot de outreach no LinkedIn"*, vende *"sistema que gera leads qualificados sem gastar em ads"*. Mesma máquina, valor visível.

Diagnose · Solve · *Value* · Price.

Quatro passos, sempre na mesma ordem. Quem inverte a ordem (vai logo para o "Price") perde — porque o cliente nunca chega a ver o valor.

01 · DIAGNOSE

Encontra a dor

Onde está o negócio a perder tempo, dinheiro ou foco?

02 · SOLVE

Constrói a cura

Sistema que resolve exatamente essa dor — nada mais.

03 · VALUE

Traduz em números

Horas poupadas, € poupados, receita ganha. Concreto.

04 · PRICE

Ancora no valor

Preço é fração do valor — não custo da solução.

EM REUNIÕES

Não comeces com *"deixa-me mostrar-te o workflow"*. Começa com perguntas: *"Onde estão a perder mais tempo? Que processos gostariam que se executassem sozinhos?"*

• EXEMPLO CONCRETO

Automação de *onboarding* de clientes.

PASSO	CONTEÚDO	NÚMERO
Diagnose	A tua equipa gasta 5 horas por semana a fazer onboarding de novos clientes.	5 h/sem.
Solve	Construo um sistema que automatiza 80% desse processo.	-80%
Value	200+ horas poupadas por ano. A \$50/h, são \$10.000 de volta no bolso, todos os anos.	\$10K/ano
Price	Estou a poupar-te \$10K/ano. Cobro \$3.000 uma única vez.	\$3K

O EFEITO

O preço deixa de ser **despesa** e passa a ser **investimento** — porque os números mostram que se paga sozinho.

Dois modelos que *funcionam*.

Não há ainda um padrão da indústria para preço de implementação de IA. Mas estes dois modelos provaram-se na prática — e combinam-se.

● ● VALUE-BASED PRICING

*Cobras pelo **impacto no negócio** — não pelas horas que demoraste. O mesmo sistema pode custar valores diferentes a empresas diferentes.*

Ideal para abrir porta com um cliente novo.

● ● MONTHLY RETAINER

*Fee mensal previsível pelo acesso contínuo à tua expertise. Tipicamente **\$1.500–\$15.000/mês**, em ciclos de 3, 6 ou 12 meses.*

Para clientes onde já provaste valor.

CONTA DE CABEÇA

Prefere **1 cliente a \$15K/mês** a **5 clientes a \$3K/mês**. Mesma receita, muito menos caos, relação mais profunda, mais referências.

- ESTRUTURAR O RETAINER

Milestones, não horas.

● HORA-A-HORA

"40 horas/mês a \$100." → Posiciona-te como *freelancer*. Discussão eterna sobre o que conta como hora.

● POR MILESTONES

"\$8K/mês · 1 sistema novo + manutenção dos atuais + 2 horas de strategy/semana." → Posiciona-te como *consultor*. Foco no resultado.

OBJETIVO DE LONGO PRAZO

Trabalhar para o papel de **Chief AI Officer** da empresa — confiável, estratégico, indispensável. Os milestones lá chegam; as horas não.

• SÍNTESE DA PARTE X

Sabe *defender* o teu número.

Sempre que disseres um preço, assume que vão perguntar: *"Como é que chegaste a esse valor?"*

Se conseguires explicar a matemática com confiança — **horas poupadas × custo/hora = \$ economizados/ano** — deixas de ser fornecedor e passas a ser parceiro de IA. E o preço deixa de ser tema.

1×

CLIENTE CERTO, BEM SERVIDO

10×

MAIS VALOR DO QUE COBRAR 5 CLIENTES MAL SERVIDOS

∞

REFERÊNCIAS E RENOVAÇÕES

PARTE XI

XI

Deploy de *automações*.

Como passar do "Cloud Code na minha máquina" para um workflow que corre todos os dias às 6h da manhã — ou cada vez que um webhook é disparado.

• DISTINÇÃO FUNDAMENTAL

Trigger *humano* vs. trigger *autónomo*.

● ● DISPARADO POR TI

"Acabei a call — escreve uma proposta para o cliente B." O agente *está ali contigo*. Pode pedir clarificação, ajustar abordagem, auto-curar erros em tempo real.

Self-healing 100% ativo.

● ● POR AGENDAMENTO OU WEBHOOK

Corre *todas as manhãs às 6h*, ou quando alguém submete um form. Já não é o agente que corre — é só o código (*workflows + tools*) que ele criou.

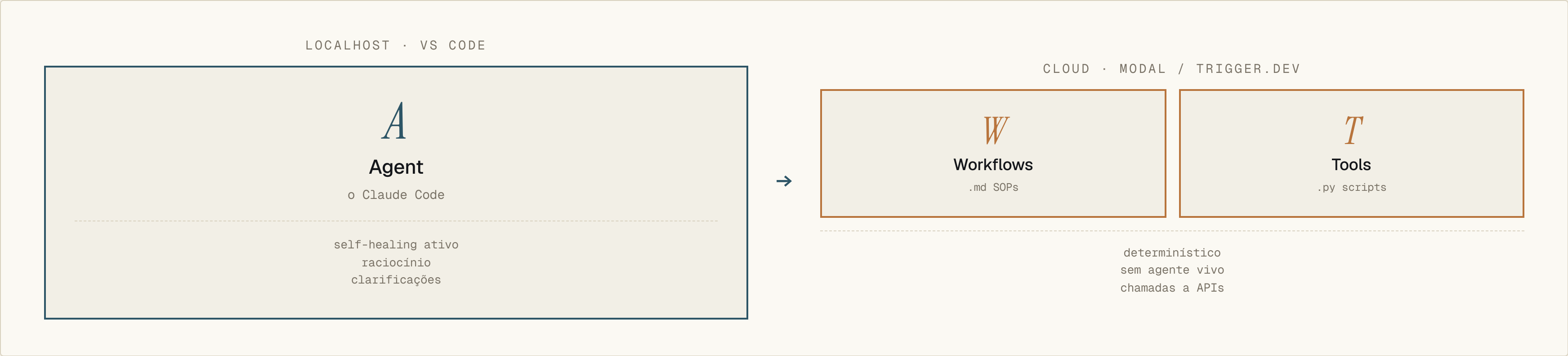
Comporta-se como automação tradicional.

CITAÇÃO DIRETA DA FONTE

"Quando fazes deploy de uma automação, **não estás a fazer deploy do agente**. Estás a fazer deploy do *workflow* e das *tools*. O agente fica no Cloud Code, localmente."

• O QUE VAI PARA A CLOUD

Do *WAT*, só *W* + *T* entram em produção.



A consequência: se um tool falha em produção, **não há um agente para se auto-corrigir**. Por isso, antes de fazer deploy, testa em modo manual até o sistema ser robusto.

- ONDE E COMO

Plataformas e *tipos de trigger*.

Plataformas mais usadas

PLATAFORMA	QUANDO USAR
Modal	Simples, focado em workloads Python. Bom 1.º deploy.
Trigger.dev	Mais flexível, UI mais limpa, multi-linguagem.
VPS próprio	Hostinger ~6 €/mês — controlo total, sempre ligado.
Vercel	Para a parte web do workflow.

Dois tipos de trigger

01 · CRON

Por agendamento

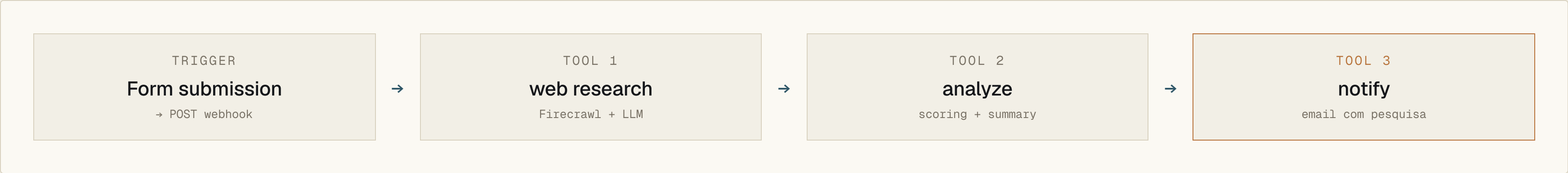
"Todas as Segundas às 6h". Cria-se com expressão cron no deploy.

02 · WEBHOOK

Por evento

Recebe um POST de outro sistema (form, Postman, app externa) e dispara o workflow.

Form preenchido → *notificação inteligente.*



CHECKLIST ANTES DE FAZER DEPLOY

Há webhooks expostos? Estão protegidos? Há segredos no código? Há limites de rate? Como vais saber se falhar?

WORKFLOW REAL

Lead "Chipotle" entra → tool faz research → recebes notificação com resumo do potencial cliente, *sem mexeres um dedo.*

PARTE XII

XII

RAG

multimodal.

Dar ao Claude Code memória pesquisável dos teus documentos, imagens, vídeos e áudio — com a nova geração de embedding models.

Retrieval · Augmented · Generation.

Um modelo só sabe o que está nos dados de treino. Quando lhe perguntas algo fora desse universo, o RAG dá-lhe três passos: **retrieve**, **augment**, **generate**.

01

Retrieve

Vai buscar a informação relevante — à tua base de dados, aos teus PDFs, às tuas imagens. Não está na cabeça do modelo, está num **vector store**.

02

Augment

Combina o que foi recuperado com a pergunta original do utilizador. Dá ao modelo *contexto* que ele não tinha.

03

Generate

O modelo gera a resposta **já informada**. Respostas factualmente ancoradas, com fontes citáveis.

• PIPELINE DE INGESTÃO TÍPICO

Como os *documentos* entram na base.



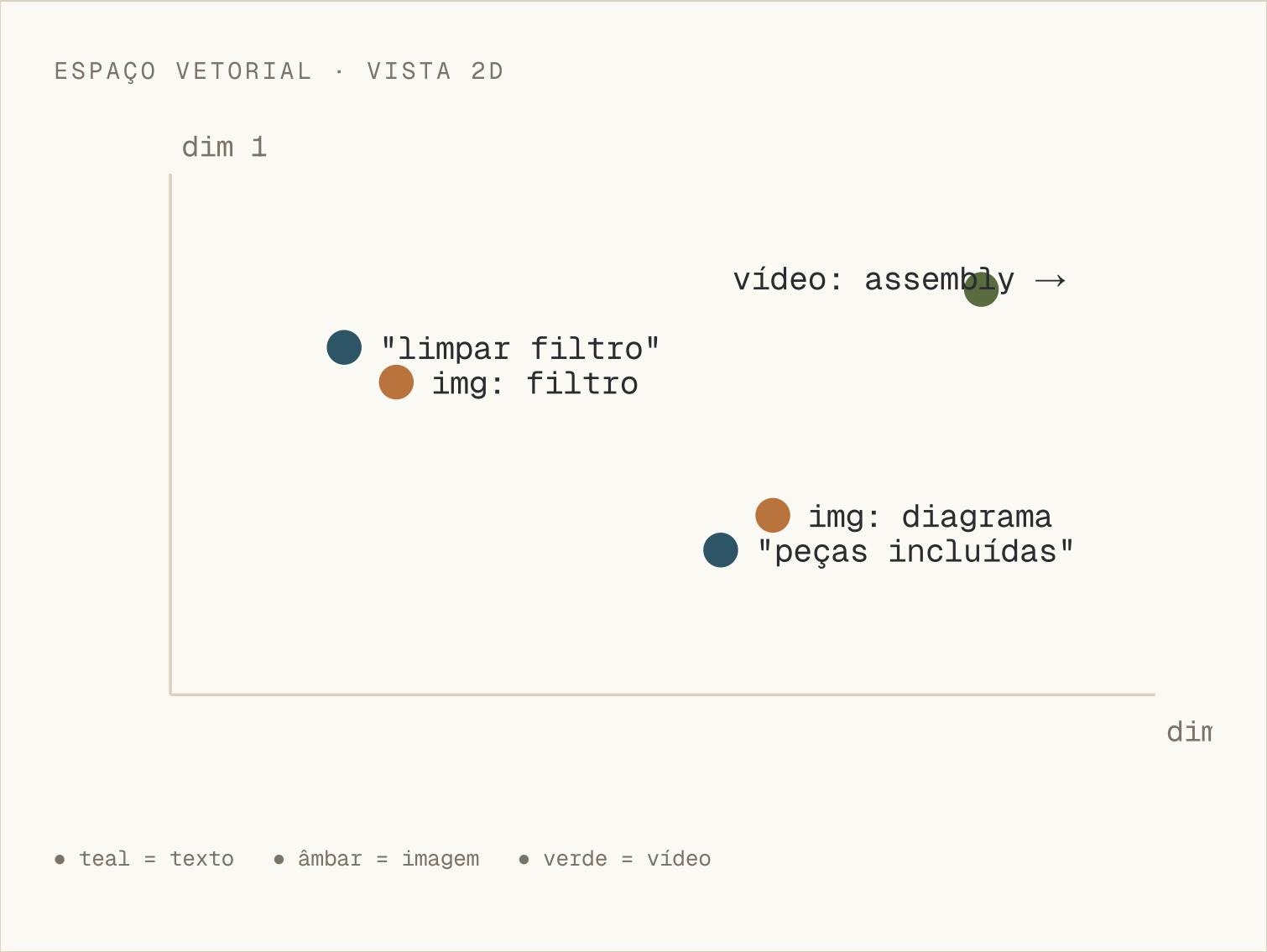
Cada **chunk** torna-se um ponto num espaço multidimensional. Pedaços com significado parecido ficam *próximos*.
Pesquisar é encontrar os vizinhos mais próximos da pergunta.

• GEMINI EMBEDDING 2

Texto, imagem, vídeo, áudio — *na mesma base.*

A geração anterior de embeddings era tipicamente *só texto*. O **Gemini Embedding 2**, da Google, foi o primeiro modelo nativamente **multimodal**: o mesmo espaço vetorial alberga frases, imagens, frames de vídeo e excertos de áudio.

Resultado: podes perguntar *"que peças tem este aspirador?"* e a base devolve simultaneamente o **parágrafo** da página 6 e a **imagem** do diagrama da página 12.



• DOIS CASOS REAIS

Dois projetos construídos em *~30 minutos*.

CASO 01

Chat com manual PDF

PDF de **68 páginas** sobre um aspirador. Texto + diagramas + imagens — todos indexados na mesma DB. A pergunta *"como limpo o filtro?"* devolve passos textuais e o diagrama correspondente. Cada resposta inclui o *match score* da página citada.

[PLACEHOLDER · screenshot da app com texto + diagrama lado-a-lado]

CASO 02

Roofing similarity

13 imagens de telhados, cada uma com metadata (custo, equipa, tempo). Cliente faz upload da foto do seu telhado → sistema devolve os **5 projetos passados mais semelhantes**, com *% match*, intervalo de orçamento, observações.

[PLACEHOLDER · grelha de 5 fotos com % match e preço de cada]

A NOTA HONESTA

O ROI não está na sofisticação técnica — é trivial. Está no **tempo de construção**: o que em n8n levaria várias horas (chunking, embeddings, retrieval, UI), aqui faz-se em ~30 min porque o agente trata de tudo.

PARTE XIII

XIII

Construir e publicar *sites*.

Como pedir landing pages que não pareçam "AI vibe-coded", mas sim profissionais — incluindo sites com animações 3D ao estilo Apple.

• CINCO HACKS DE QUEM DOMINA O OFÍCIO

Para sites que *não pareçam* AI.

- 01

0. CLAUDE.md específico para web — pré-requisito. Coloca regras "sempre invocar front-end design skill" no topo do ficheiro.
- 02

1. Front-end design skill — skill da Anthropic adaptada que ensina o Claude a escolher tipografia, espaçamento, hierarquia e cores antes de escrever HTML.
- 03

2. Sistema visual antes do código — peças paleta + tipografia + grelha + componentes *primeiro*. O HTML é o último passo, não o primeiro.
- 04

3. Referências reais — junta screenshots de sites que admiras. O Claude analisa-os para extrair vocabulário visual concreto, não genérico.
- 05

4. Iteração componente a componente — não peças "o site inteiro". Pede hero, depois sec. features, depois testemunhos. Cada um afinado antes do próximo.

• A SKILL QUE MUDA TUDO

Front-end design *skill*.

É um ficheiro Markdown que diz ao Claude **como pensar em design** antes de codificar — paletas harmoniosas, hierarquia tipográfica, ritmo, espaço em branco, animações subtis.

Faz parte das skills oficiais da Anthropic. Quando ativa, o Claude deixa de gerar HTML genérico e começa a fazer *escolhas de design* deliberadas.

COMO ATIVAR SEMPRE

Pôr no CLAUDE.md: *"Always invoke the front-end design skill before writing any front-end code every session. No exceptions."*

CLAUDE.md (extracto)

Front-end rules

- Sempre invocar a skill `frontend-design` antes de escrever qualquer JSX/HTML.
- Paleta: 1 dominante, 2 acentos máx. Sem mais.
- Tipografia: 1-2 families.
- Animações: subtis, ≤ 300 ms.
- Nunca usar `Inter` nem gradientes "AI typical".

• VÍDEO → LANDING PAGE

Sites ao estilo *Apple* em ~30 min.

Com duas skills (*front-end design + video-to-website*) dá um **vídeo de um produto** ao Claude e diz *"build me a website for this video"*.

Sem dares copy, sem dares paleta, sem dares info do produto — recebes uma landing page com:

- Hero com produto destacado
- Animação ao scroll (camera explode em peças)
- Texto dinâmico que aparece em sync
- Paleta extraída do próprio vídeo

PIPELINE DA SKILL

1. `analyze` video frames
2. `extract` palette + motion
3. `draft` copy from product
4. `build` HTML/CSS/JS
5. `wire` scroll-driven anim
6. `preview` + iterate

[PLACEHOLDER · frame do site
com produto a desmontar-se ao scroll]

PARTE XIV

XIV

Google Workspace *CLI*.

Como dar ao Claude Code acesso completo ao teu Gmail, Drive, Docs, Sheets, Calendar e Slides — através de uma única interface de linha de comandos.

- O QUE É O GWS CLI

Uma *única ferramenta*, todo o Google Workspace.

Projeto **open-source da Google** que expõe todo o Workspace como CLI. Não é (ainda) um produto oficial suportado — mas é um Google project, ativo no GitHub.

Em vez de configurar 6 MCP servers (um por serviço) com tools que enchem o contexto, dás ao Claude **uma só CLI**. Ele descobre comandos sozinho.

NOTA DA FONTE

"This is not an officially supported Google product (...) under active development. Expect breaking changes as we march towards v1.0."

Benefícios principais

01

One interface

Gmail, Drive, Docs, Sheets, Calendar, Slides, Admin — uma só CLI.

02

JSON-first

Respostas estruturadas, ideal para IA processar.

03

Auto-discovery

Endpoints novos da Google apanhados automaticamente.

04

Recipes integradas

100+ skills pré-feitas para tarefas comuns.

• SETUP EM 6 PASSOS

Autenticar via *OAuth*.

- 01 Pedir ao Claude para instalar a GWS CLI a partir do **GitHub repo oficial** da Google.
- 02 Criar projeto no **Google Cloud Console** (ex.: "Claude Code GWS").
- 03 Configurar **OAuth consent screen** (interno ou externo + test users).
- 04 Gerar **OAuth Client ID** tipo *Desktop app* e descarregar o JSON.
- 05 Colocar o ficheiro em `~/.config/gws/client_secret.json`.
- 06 Correr `gws auth login` → escolher conta → autorizar APIs.

TRUQUE

Para cada Workspace API que queiras usar (Gmail, Drive, Docs...) tens de a **ativar** no projeto da Cloud Console. O Claude indica quais faltam quando uma chamada falha.

Tornar a inbox *auto-priorizada*.

Pedido em linguagem natural ao Claude:

"Vai buscar os emails não lidos de hoje. Com base no que sabes do meu negócio e prioridades, atribui um priority score a cada um. Os abaixo de 5, marca-os como lidos automaticamente."

Output real (30 emails analisados):

→ Emails ordenados com score (0–10)

→ Justificação por email

→ Os abaixo de 5 marcados como lidos

→ Resumo do que requer ação humana

OUTROS CASOS DO MESMO CURSO

→ YouTube link → Google Doc formatado (header, imagem, links, CTA)

→ Pesquisa em Drive: "encontra Docs feitos em Abril de 2025"

→ Geração automática de Slides

→ Criar eventos no Calendar a partir de Sheets

→ Labels + archive automáticos

[PLACEHOLDER · screenshot real
do output da triagem com scores]

- POR EXPLORAR – PRÓXIMOS VOLUMES

E nas *outras clouds*?

A lógica do GWS CLI — uma só ferramenta, um só auth, descoberta automática de comandos, JSON-first — replica-se em qualquer ecossistema cloud. Dois candidatos naturais para volumes futuros:

PRÓXIMO VOLUME

AWS CLI

EC2, S3, Lambda, RDS, IAM, CloudWatch — o Claude Code a operar a tua stack AWS via `aws`. Equivalente ao GWS, mas para a maior cloud pública.

PRÓXIMO VOLUME

Azure CLI

App Services, Functions, Key Vault, AAD, Storage — todo o Microsoft Azure controlado a partir do Claude Code via `az`.

PADRÃO A REPETIR

Os passos de setup serão similares: instalar CLI → criar credenciais/service principal → autenticar → enable APIs/services → primeiro comando de teste. Em ambos, a regra de ouro mantém-se: **permissões mínimas**.

PARTE XV

XV

Assistente executivo *próprio.*

Como combinar tudo o que aprendeste — CLAUDE.md, skills, sub-agents, GWS CLI — para construir um "segundo cérebro" que sabe tudo sobre ti e o teu negócio.

• CONSTRUÇÃO EM QUATRO FASES

Casa · Vida · Mãos · Crescimento.

FASE 01

Casa

A **estrutura**. Pastas, subpastas, convenções de nomes. Onde vivem skills, sub-agents, contexto do negócio.

FASE 02

Vida

As **regras + contexto**. Quem és, o que faz a tua empresa, prioridades, equipa, OKRs do trimestre.

FASE 03

Mãos

As **skills**. Primeira skill, depois mais skills e sub-agents — passa de pensar a *fazer*.

FASE 04

Crescimento

A **melhoria contínua**. Após cada execução, refinar workflows. Adicionar memória. Vai ficando mais útil com o tempo.

• ANATOMIA DO "HERK 2"

Como o *autor* organiza o seu.

```
herk-2/
├─ CLAUDE.md # quem sou, o que faço
├─ .claude/
│   └─ skills/
│       ├── morning_coffee/
│       ├── pulse_check/
│       ├── carousel/
│       ├── excalidraw_visuals/
│       └─ youtube_comment_audit/
├─ current/ # prioridades · OTAAS
├─ team/
├─ projects/
│   ├── carousels/<data>/
│   └─ visualizations/<data>/
├─ decisions/
└─ temp/
```

Skills reais do Herk 2

morning_coffee	Lê agenda + ClickUp + objetivos → propõe plano do dia.
pulse_check	Estado de cada iniciativa + follow-ups que escaparam.
carousel	Transcrição → 7-8 slides no estilo do autor.
excalidraw_visuals	Diagramas com texto sempre correto (typing, não AI).
comment_audit	Scrape de comentários do YouTube + análise.

• O QUE ISTO DESBLOQUEIA

Quatro sessões em paralelo, *~1 minuto*.

SESSÃO 01 Morning coffee Plano do dia, agenda bloqueada, decisões prontas para "Sim".	SESSÃO 02 Sub-agent · pesquisa Investiga feature nova → entrega LinkedIn post + carousel de 8 slides.	SESSÃO 03 Pulse check Estado de cada iniciativa do trimestre + 3 follow-ups que escaparam.	SESSÃO 04 Visualização Excalidraw com texto perfeito para o próximo vídeo.
--	--	---	---

~1 min TEMPO A DESPACHAR 4 TAREFAS	~25 min TEMPO MANUAL SEM ASSISTENTE	25× MULTIPLICADOR DE OUTPUT
--	---	---

• POR EXPLORAR – PRÓXIMOS VOLUMES

O *passo seguinte*?

O assistente pessoal do Nate Herk é um excelente ponto de partida. O passo lógico seguinte é desenhar um *sistema operativo* de IA sobre o Claude Code, com convenções, módulos e workflows reutilizáveis à escala da organização.

PRÓXIMO VOLUME

DAT Claude OS

Um modelo arquitetural — convenções de pastas, taxonomia de skills, contratos entre sub-agents, padrões de memória partilhada, governança e fluxos de instalação — que transforma o assistente pessoal num **OS de IA reutilizável e auditável** ao nível da equipa.

[PLACEHOLDER · diagrama de arquitetura do DAT Claude OS
módulos · skills shared · sub-agents · memória · governance]

FIM DA PARTE XV

Conhecimento arrumado.

Falta o mais importante: pôr as mãos em código, lançar o primeiro workflow, e ir aprendendo com cada erro. As próximas três páginas fecham este volume.

• RECAP

O que *levas contigo*.

I Fundamentos Custo, prompting, modos, modelos, tokens.	II CLAUDE.md & arq. Slash commands e <code>.claude/</code>.	III Gestão de contexto Oito práticas anti-desperdício.	IV GitHub & Worktrees Sessões paralelas.	V Onde correr Terminal, Desktop, Web, IDE, VPS.
VI WAT framework Workflows, Agent, Tools.	VII Skills · Sub-ag. · Teams Alavancagem multiplicada.	VIII APIs · MCPs · Browser Superpoderes externos.	IX Hacks Atalhos de quem domina.	X Monetizar Vender soluções, não tools.
XI Deploy Cron + webhook · Modal/Trigger.	XII RAG multimodal Texto, imagem, vídeo, áudio.	XIII Sites & 3D Anti vibe-coded, com animações.	XIV Google CLI Todo o Workspace numa CLI.	XV Exec. Assistant Casa · Vida · Mãos · Crescimento.

• O QUE AINDA FICA POR COBRIR

O que *não coube*.

Esta edição cobre os 10 blocos essenciais. O curso completo do Nate Herk continua com temáticas mais concretas — builds passo a passo — que esta tradução pode evoluir para incluir.

07	Deploy de automações ○	13	Build/deploy de websites ○	14	Sites 3D animados ○
10	RAG ○	16	Google CLI ○	17	Executive Assistant ○
11	n8n workflow → app ○	26	Encontrar clientes ○	27	Primeiro cliente em 7 dias ○
29	Entregar projetos AI ○	–	Recursos & comunidade ○	–	Casos práticos avançados ○

• ATRIBUIÇÃO

Sobre *esta edição*.

Fontes

Esta versão portuguesa é uma **adaptação derivada** de duas fontes do **Nate Herk** (AI Automation Society):

→ **Claude Code Masterclass** (PDF de slides original — ~64 sl.)

→ Curso *Build & Sell with Claude Code* em vídeo (~10h, transcrição completa)

O conteúdo pedagógico, a estrutura dos capítulos e os exemplos práticos são da responsabilidade do autor original. Esta edição traduz, contextualiza e re-formata para um público de língua portuguesa.

Esta edição

Variante linguística: Português de Portugal (pt-PT).
Tom: formal/técnico, registo de manual profissional.
Código e comandos: mantidos em inglês para preservar o uso real.
Design: sistema original — não replica a identidade visual da Anthropic nem do material original.

MARCAS REGISTRADAS

“Claude”, “Claude Code” e logótipos associados são marcas da **Anthropic, PBC**. Referenciados apenas para fins didáticos.

FIM DO VOLUME

Obrigado. Agora *vai construir.*

O Claude Code é uma ferramenta. O valor está no que constróis com ela — e na disciplina com que geres contexto, planeias fases e usas as práticas que aqui vimos.